

ChatGPT ▾

Log

The problem that we want to solve is cycling through old employees and replacing them with new hires. However, we want to preserve the first 3 hires as those are the founders of the company. We want to use one stack and one queue, all using OOP principles. All of the code should be on one page, using Java.

A clean OOP approach is:

- Store the **founders** (first 3 hires) separately so they are never removed.
- Store all other employees in a **queue** (FIFO) to represent hiring order.
- Use a **stack** to keep track of employees who were replaced (LIFO history).
- When cycling employees:
 1. Remove the oldest non-founder from the queue.
 2. Push that employee onto the stack.
 3. Add the new hire to the queue.

This satisfies the requirement of using **one queue**, **one stack**, and **OOP principles** (encapsulation through classes).

```
</> Java   
  
import java.util.*;  
  
class Employee {  
    private String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    @Override  
    public String toString() {  
        return name;  
    }  
}  
  
class Company {  
    private List<Employee> founders;  
    private Queue<Employee> employees;  
    private Stack<Employee> replacedEmployees;  
  
    public Company() {  
        founders = new ArrayList<>();  
        employees = new LinkedList<>();  
        replacedEmployees = new Stack<>();  
    }  
  
    // Hire an employee  
    public void hire(Employee employee) {  
        if (founders.size() < 3) {  
            founders.add(employee);  
        } else {  
            employees.offer(employee);  
        }  
    }  
}
```

```

// Replace the oldest non-founder with a new hire
public void replaceEmployee(Employee newHire) {
    if (!employees.isEmpty()) {
        Employee removed = employees.poll();
        replacedEmployees.push(removed);

        System.out.println("Replaced: " + removed +
            " with " + newHire);

        employees.offer(newHire);
    } else {
        System.out.println("No non-founder employees available to replace.");
    }
}

public void displayEmployees() {
    System.out.println("\nCurrent Employees:");

    System.out.println("Founders:");
    for (Employee founder : founders) {
        System.out.println("  " + founder);
    }

    System.out.println("Other Employees:");
    for (Employee employee : employees) {
        System.out.println("  " + employee);
    }
}

public void displayReplacementHistory() {
    System.out.println("\nReplacement History (Most Recent First):");
    for (int i = replacedEmployees.size() - 1; i >= 0; i--) {
        System.out.println("  " + replacedEmployees.get(i));
    }
}

}

public class Main {
    public static void main(String[] args) {

        Company company = new Company();

        // First 3 hires = founders
        company.hire(new Employee("Alice"));
        company.hire(new Employee("Bob"));
        company.hire(new Employee("Charlie"));

        // Regular employees
        company.hire(new Employee("David"));
        company.hire(new Employee("Eve"));
        company.hire(new Employee("Frank"));

        company.displayEmployees();

        // Cycle employees
        company.replaceEmployee(new Employee("Grace"));
        company.replaceEmployee(new Employee("Henry"));

        company.displayEmployees();
        company.displayReplacementHistory();
    }
}

```

Example Output

Current Employees:
Founders:
Alice
Bob
Charlie
Other Employees:
David
Eve
Frank

Replaced: David with Grace
Replaced: Eve with Henry

Current Employees:
Founders:
Alice
Bob
Charlie
Other Employees:
Frank
Grace
Henry

Replacement History (Most Recent First):
Eve
David



Time Complexity

- Hire: $O(1)$
- Replace employee: $O(1)$ (queue dequeue/enqueue + stack push)

 Monday.com · Sponsored

...

If you need a task manager

See how teams manage tasks faster with AI on monday.com.



To support this new rule, each non-founder employee needs to track how many **replacement cycles** they have survived.

New Behavior

- First 3 hires are founders (permanent immediately).
- Every other hire starts in the replaceable queue.
- During each replacement cycle:
 - The oldest replaceable employee is removed.
 - Everyone else in the queue gains **+1 cycle survived**.
 - Any employee whose cycle count reaches **3** is promoted to the permanent group.
 - The new hire enters the queue with 0 cycles survived.

This creates a growing set of permanent employees over time.

```
</> Java

import java.util.*;

class Employee {
    private String name;
    private int cyclesSurvived;

    public Employee(String name) {
        this.name = name;
        this.cyclesSurvived = 0;
    }

    public String getName() {
        return name;
    }

    public int getCyclesSurvived() {
        return cyclesSurvived;
    }

    public void incrementCycles() {
        cyclesSurvived++;
    }

    @Override
    public String toString() {
        return name + " (Cycles: " + cyclesSurvived + ")";
    }
}

class Company {

    // Permanent employees (founders + promoted employees)
    private List<Employee> permanentEmployees;

    // Replaceable employees
    private Queue<Employee> employeeQueue;

    // History of replaced employees
    private Stack<Employee> replacedEmployees;

    public Company() {
        permanentEmployees = new ArrayList<>();
        employeeQueue = new LinkedList<>();
        replacedEmployees = new Stack<>();
    }

    public void hire(Employee employee) {
        if (permanentEmployees.size() < 3) {
            permanentEmployees.add(employee); // founders
        } else {
            employeeQueue.offer(employee);
        }
    }

    public void replaceEmployee(Employee newHire) {

        if (employeeQueue.isEmpty()) {
            System.out.println("No replaceable employees available.");
            return;
        }

        // Remove oldest replaceable employee
        Employee removed = employeeQueue.poll();
        replacedEmployees.push(removed);
    }
}
```

```
        System.out.println("Replaced: " + removed.getName()
            + " with " + newHire.getName());

        // Everyone remaining survives one more cycle
        int size = employeeQueue.size();

        for (int i = 0; i < size; i++) {
            Employee current = employeeQueue.poll();

            current.incrementCycles();

            if (current.getCyclesSurvived() >= 3) {
                permanentEmployees.add(current);

                System.out.println(
                    current.getName()
                    + " has survived 3 cycles and is now permanent!");
            }
            else {
                employeeQueue.offer(current);
            }
        }

        // Add new hire to replaceable queue
        employeeQueue.offer(newHire);
    }

    public void displayEmployees() {

        System.out.println("\n==== PERMANENT EMPLOYEES =====");
        for (Employee employee : permanentEmployees) {
            System.out.println(employee.getName());
        }

        System.out.println("\n==== REPLACEABLE EMPLOYEES =====");
        for (Employee employee : employeeQueue) {
            System.out.println(employee);
        }
    }

    public void displayReplacementHistory() {

        System.out.println("\n==== REPLACEMENT HISTORY =====");

        for (int i = replacedEmployees.size() - 1; i >= 0; i--) {
            System.out.println(replacedEmployees.get(i).getName());
        }
    }
}

public class Main {

    public static void main(String[] args) {

        Company company = new Company();

        // Founders
        company.hire(new Employee("Alice"));
        company.hire(new Employee("Bob"));
        company.hire(new Employee("Charlie"));

        // Initial employees
        company.hire(new Employee("David"));
        company.hire(new Employee("Eve"));
        company.hire(new Employee("Frank"));

        company.displayEmployees();
    }
}
```

```
company.replaceEmployee(new Employee("Grace"));
company.replaceEmployee(new Employee("Henry"));
company.replaceEmployee(new Employee("Ivy"));
company.replaceEmployee(new Employee("Jack"));

company.displayEmployees();
company.displayReplacementHistory();
}
}
```

Example Progression

Initial queue:

David, Eve, Frank



Cycle 1:

David removed
Eve(1), Frank(1), Grace(0)



Cycle 2:

Eve removed
Frank(2), Grace(1), Henry(0)



Cycle 3:

Frank removed
Grace(2), Henry(1), Ivy(0)



Cycle 4:

Grace survives her 3rd cycle
Grace becomes permanent
Henry(2), Ivy(1), Jack(0)



Permanent employees:

Alice
Bob
Charlie
Grace



This keeps exactly **one queue** for replaceable employees and **one stack** for replacement history while promoting employees to the permanent list after surviving three replacement cycles.

Alter the founders to be 4 starting in number

That's a very small change. Anywhere the code checks whether an employee should be treated as a founder, change the limit from **3** to **4**.

Replace the `hire()` method with:

```
</> Java
public void hire(Employee employee) {
    if (permanentEmployees.size() < 4) { // First 4 hires are founders
        permanentEmployees.add(employee);
    } else {
        employeeQueue.offer(employee);
    }
}
```

And update the comments in `main()` :

```
</> Java
// Founders
company.hire(new Employee("Alice"));
company.hire(new Employee("Bob"));
company.hire(new Employee("Charlie"));
company.hire(new Employee("Diana"));
```

Then add the remaining employees:

```
</> Java
company.hire(new Employee("David"));
company.hire(new Employee("Eve"));
company.hire(new Employee("Frank"));
```

New Initial State

Permanent Employees (Founders):

```
Alice
Bob
Charlie
Diana
```

Replaceable Queue:

```
David (0)
Eve (0)
Frank (0)
```

Everything else in the program remains the same:

- First **4 hires** are permanent founders.
- Later hires enter the queue.
- Employees surviving **3 replacement cycles** are promoted to permanent status.
- Replaced employees are stored in the stack history.
- One queue and one stack are still used.

send the entire code back to me with the changes please

Here's the complete Java program with:

- **4 founders** (permanent from the start)
- **1 Queue** for replaceable employees
- **1 Stack** for replacement history
- Employees become **permanent after surviving 3 replacement cycles**
- OOP design using `Employee` and `Company` classes

```
<> Java   
  
import java.util.*;  
  
class Employee {  
    private String name;  
    private int cyclesSurvived;  
  
    public Employee(String name) {  
        this.name = name;  
        this.cyclesSurvived = 0;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getCyclesSurvived() {  
        return cyclesSurvived;  
    }  
  
    public void incrementCycles() {  
        cyclesSurvived++;  
    }  
  
    @Override  
    public String toString() {  
        return name + " (Cycles Survived: " + cyclesSurvived + ")";  
    }  
}  
  
class Company {  
  
    // Permanent employees (founders + promoted employees)  
    private List<Employee> permanentEmployees;  
  
    // Replaceable employees  
    private Queue<Employee> employeeQueue;  
  
    // Employees that have been replaced  
    private Stack<Employee> replacedEmployees;  
  
    public Company() {  
        permanentEmployees = new ArrayList<>();  
        employeeQueue = new LinkedList<>();  
        replacedEmployees = new Stack<>();  
    }  
  
    // Hire employee  
    public void hire(Employee employee) {  
  
        // First 4 hires are founders
```



```

        if (permanentEmployees.size() < 4) {
            permanentEmployees.add(employee);
            System.out.println(employee.getName() + " hired as a Founder.");
        } else {
            employeeQueue.offer(employee);
            System.out.println(employee.getName() + " added to replaceable employees.");
        }
    }

    // Replace oldest replaceable employee
    public void replaceEmployee(Employee newHire) {

        if (employeeQueue.isEmpty()) {
            System.out.println("No replaceable employees available.");
            return;
        }

        // Remove oldest employee
        Employee removed = employeeQueue.poll();
        replacedEmployees.push(removed);

        System.out.println("\nReplaced: " + removed.getName()
            + " with " + newHire.getName());

        // Process remaining employees
        int size = employeeQueue.size();

        for (int i = 0; i < size; i++) {

            Employee current = employeeQueue.poll();

            current.incrementCycles();

            if (current.getCyclesSurvived() >= 3) {

                permanentEmployees.add(current);

                System.out.println(current.getName()
                    + " survived 3 cycles and is now PERMANENT.");

            } else {

                employeeQueue.offer(current);
            }
        }

        // Add new hire to queue
        employeeQueue.offer(newHire);
    }

    public void displayEmployees() {

        System.out.println("\n==== PERMANENT EMPLOYEES =====");

        for (Employee employee : permanentEmployees) {
            System.out.println(employee.getName());
        }

        System.out.println("\n==== REPLACEABLE EMPLOYEES =====");

        if (employeeQueue.isEmpty()) {
            System.out.println("None");
        } else {
            for (Employee employee : employeeQueue) {
                System.out.println(employee);
            }
        }
    }
}

```

```
public void displayReplacementHistory() {

    System.out.println("\n==== REPLACEMENT HISTORY =====");

    if (replacedEmployees.isEmpty()) {
        System.out.println("No employees have been replaced.");
        return;
    }

    for (int i = replacedEmployees.size() - 1; i >= 0; i--) {
        System.out.println(replacedEmployees.get(i).getName());
    }
}

public class Main {

    public static void main(String[] args) {

        Company company = new Company();

        // 4 Founders
        company.hire(new Employee("Alice"));
        company.hire(new Employee("Bob"));
        company.hire(new Employee("Charlie"));
        company.hire(new Employee("Diana"));

        // Regular employees
        company.hire(new Employee("David"));
        company.hire(new Employee("Eve"));
        company.hire(new Employee("Frank"));

        company.displayEmployees();

        // Replacement cycles
        company.replaceEmployee(new Employee("Grace"));
        company.replaceEmployee(new Employee("Henry"));
        company.replaceEmployee(new Employee("Ivy"));
        company.replaceEmployee(new Employee("Jack"));

        company.displayEmployees();

        company.displayReplacementHistory();
    }
}
```

Expected Result After the 4 Replacements

Permanent Employees

Alice
Bob
Charlie
Diana
Grace



Replaceable Employees

Henry (Cycles Survived: 2)
Ivy (Cycles Survived: 1)
Jack (Cycles Survived: 0)



Replacement History

Grace
Frank
Eve
David



In this example, Grace survives three replacement cycles and is automatically promoted to the permanent employee list alongside the four founders.